CHAINLINK CRE DATA FEEDS PAIRVAULT UNISWAP LP

LPFY

Chainlink & CRE in the protocol

How LPFY uses **Chainlink Data Feeds** to price Uniswap LP collateral and **Chainlink CRE** as an automated liquidation keeper.

Summary · Chainlink surface · Data Feeds · CRE keeper · End-to-end · On-chain · How the keeper works · Repo paths · Hackathon demo · Security model · Deployed

---

1. Summary

> Chainlink Data Feeds price Uniswap LP collateral for LTV and health checks. Chainlink CRE runs as an automated liquidation keeper: it detects underwater loans and triggers on-chain unwind, then recovers USDC into the pair's ERC-4626 PairVault.

PROBLEM

Loans go underwater

When collateral value falls, `liquidate` must run reliably — without opening liquidation to everyone.

CRE

Automated upkeep

Cron → check for liquidatable loans → signed report → on-chain `liquidate`.

FEEDS

Collateral pricing

ETH/USD, USDC/USD, and BTC/USD feeds drive position value for LTV and liquidation health.

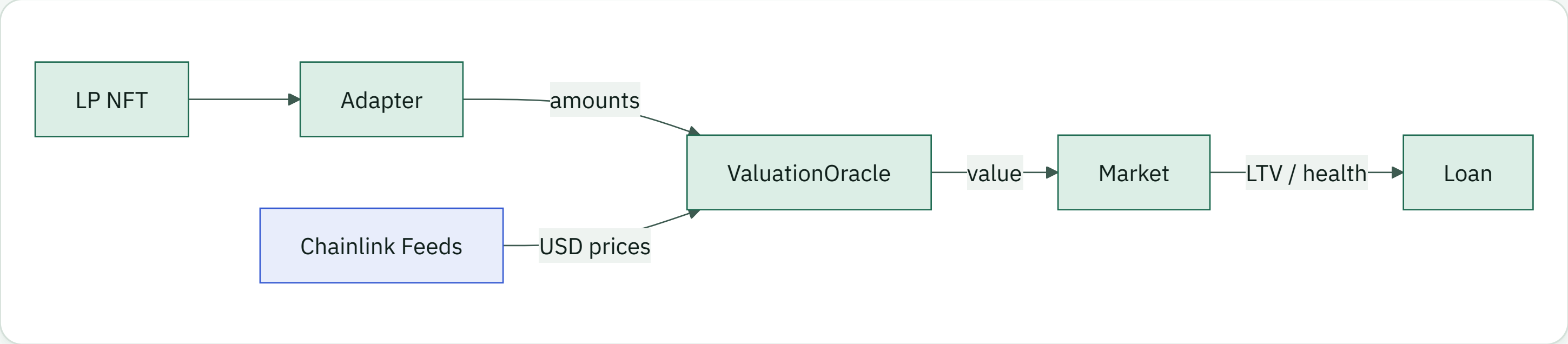
2. Chainlink surface

PRODUCT	ROLE IN LPFY
Data Feeds	USD prices for LP valuation, LTV, and liquidatable checks
CRE	Automated liquidation keeper (schedule → read → write report)
Keystone Forwarder	Delivers signed CRE reports to the on-chain receiver

> Data Feeds decide *when* a loan is liquidatable. CRE decides *who/how* liquidation is triggered automatically.

3. Data Feeds (valuation)

Uniswap LP NFTs are valued from on-pool token amounts × Chainlink USD prices. The market uses that value for borrow caps, health, and liquidation thresholds.



Feeds price tokens; adapters supply LP amounts; market enforces LTV.

FEEDS

Prices used

ETH/USD · USDC/USD · BTC/USD — wired on the valuation oracle.

ORACLE

What it returns

Position value in USD, converted to debt-asset units for LTV and liquidation checks.

4. CRE liquidation keeper

CRE is not used for lend/borrow UX. Its only job is **automated liquidation upkeep**.

READ

needsUpkeep

Asks the market if any loan is underwater and returns the first loan id.

WRITE

writeReport

Sends a signed report with that loan id through the Keystone Forwarder.

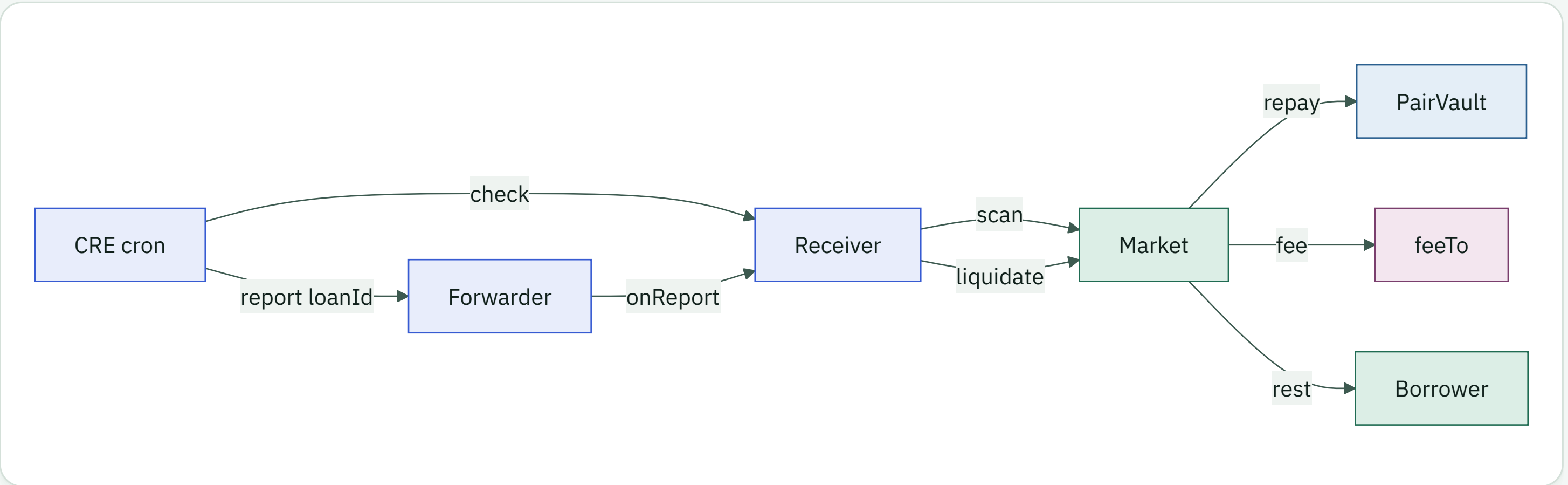
EXECUTE

liquidate

Receiver calls the market. Debt returns to PairVault; fee and surplus are split.

> After liquidate: debt USDC is pushed into the pair's PairVault; `liquidationFeeBps` of surplus → `feeTo`; remainder → borrower. CRE does not configure fees.

5. End-to-end liquidation



Only the market owner or the authorized CRE receiver may execute liquidate.

1. Lender funds the PairVault; borrower posts an LP NFT and borrows near LTV.
2. Loan becomes liquidatable (price move or lower liquidation threshold).
3. CRE cron finds the loan via `needsUpkeep`.
4. CRE writes a report; forwarder delivers it to the receiver.
5. Market unwinds the NFT, swaps to USDC, repays the vault, then splits surplus.

6. On-chain surface

RECEIVER

CreLiquidationReceiver

- `needsUpkeep()` — CRE read
- `onReport` — CRE write (Forwarder only)
- Decodes loan id and calls `liquidate`

MARKET

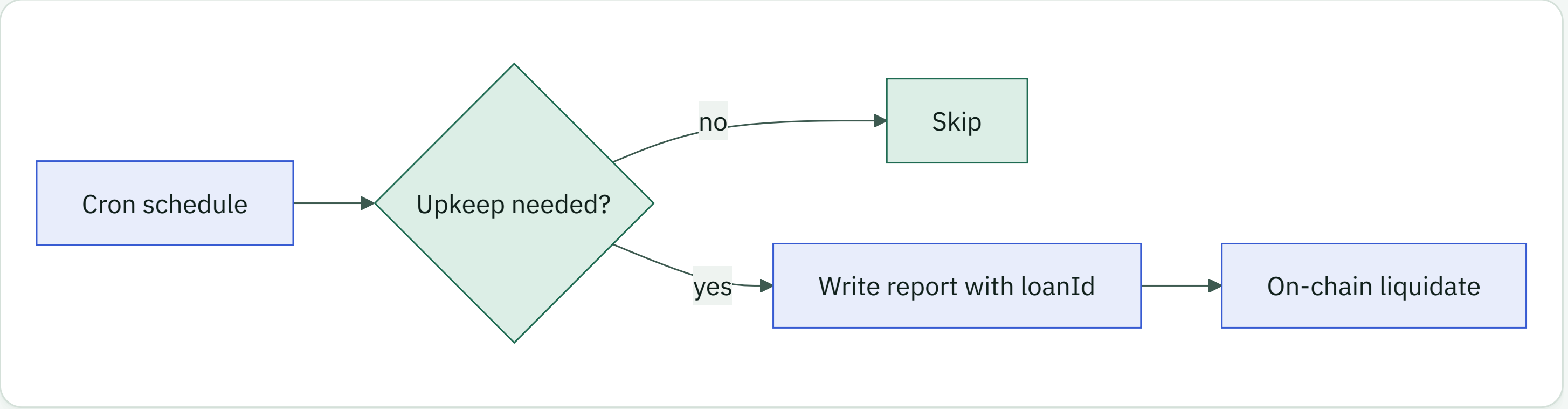
MarketLendingModule

- Scans for liquidatable loans
- `liquidate` — owner or authorized liquidator
- Receiver must be authorized

```
// Receiver (simplified)

function needsUpkeep() external view returns (bool, uint256) {
    return market.findFirstLiquidatableLoan();
}

function onReport(bytes calldata /* metadata */, bytes calldata report) external {
    require(msg.sender == forwarder);
    uint256 loanId = abi.decode(report, (uint256));
    market.liquidate(loanId);
}
```

7. How the keeper works

The CRE workflow uses a cron trigger plus an EVM client: read upkeep, then write a report when a loan is found.



BEHAVIOR

- Runs on a fixed schedule (for example every few minutes).
- If no loan is liquidatable, the run does nothing.
- If a loan is found, CRE submits a signed report that triggers liquidation.

CRE-SDK

Built from the official keeper-bot TypeScript template. Capabilities used: **Cron** + **EVMClient** (read + writeReport).

```
// cre/liquidation-keeper/workflow.ts (core)

export const onCronTrigger = (runtime) => {
  const receiver = new CreLiquidationReceiver(evmClient, contractAddress)

  const { upkeepNeeded, loanId } = receiver.needsUpkeep(runtime)
  if (!upkeepNeeded || loanId === 0n) return 'Skipped — no upkeep needed'

  const reportData = encodeAbiParameters(
    parseAbiParameters('uint256 loanId'),
    [loanId],
  )

  const writeResult = receiver.writeReport(runtime, reportData, {
    gasLimit: '3500000',  // unwind + swaps need headroom
  })
  // TxStatus.SUCCESS + receiverContractExecutionStatus OK
  return `Liquidated loan ${loanId} — tx: ${txHash}`
}

export function initWorkflow(config) {
  const cron = new cre.capabilities.CronCapability()
  return [cre.handler(cron.trigger({ schedule: config.schedule }), onCronTrigger)]
}
```

Config (`config.staging.json`):

```
{
  "schedule": "0 */5 * * * *",
  "evms": [{
    "chainSelectorName": "ethereum-testnet-sepolia",
    "contractAddress": "0x32866b81408d0260259BC0109b9F0a300Cdca132"
  }]
}
```

8. Where CRE lives in the repo

PATH	ROLE
`cre/`	CRE project root (`project.yaml`, secrets, CLI)
`cre/liquidation-keeper/workflow.ts`	Cron handler — EVM read + `writeReport`
`cre/liquidation-keeper/config.staging.json`	Schedule + receiver `contractAddress`
`smart-contracts/contracts/markets/CreLiquidationReceiver.sol`	`needsUpkeep` + `onReport`
`smart-contracts/contracts/markets/MarketLendingModule.sol`	`liquidate` / loan scan (authorized only)
`smart-contracts/contracts/ValuationOracle.sol`	LP value × Chainlink Data Feeds
`smart-contracts/deployments/sepolia-markets.json`	Live Sepolia addresses
`cre/README.md`	Simulate / broadcast / deploy runbook

Repo: github.com/unblocktechie/LPFy · CRE pattern from keeper-bot TypeScript template

> CRE is **not** used for lending/borrowing UX. Users lend & borrow via the React app. CRE's job is only **automated liquidation upkeep**.

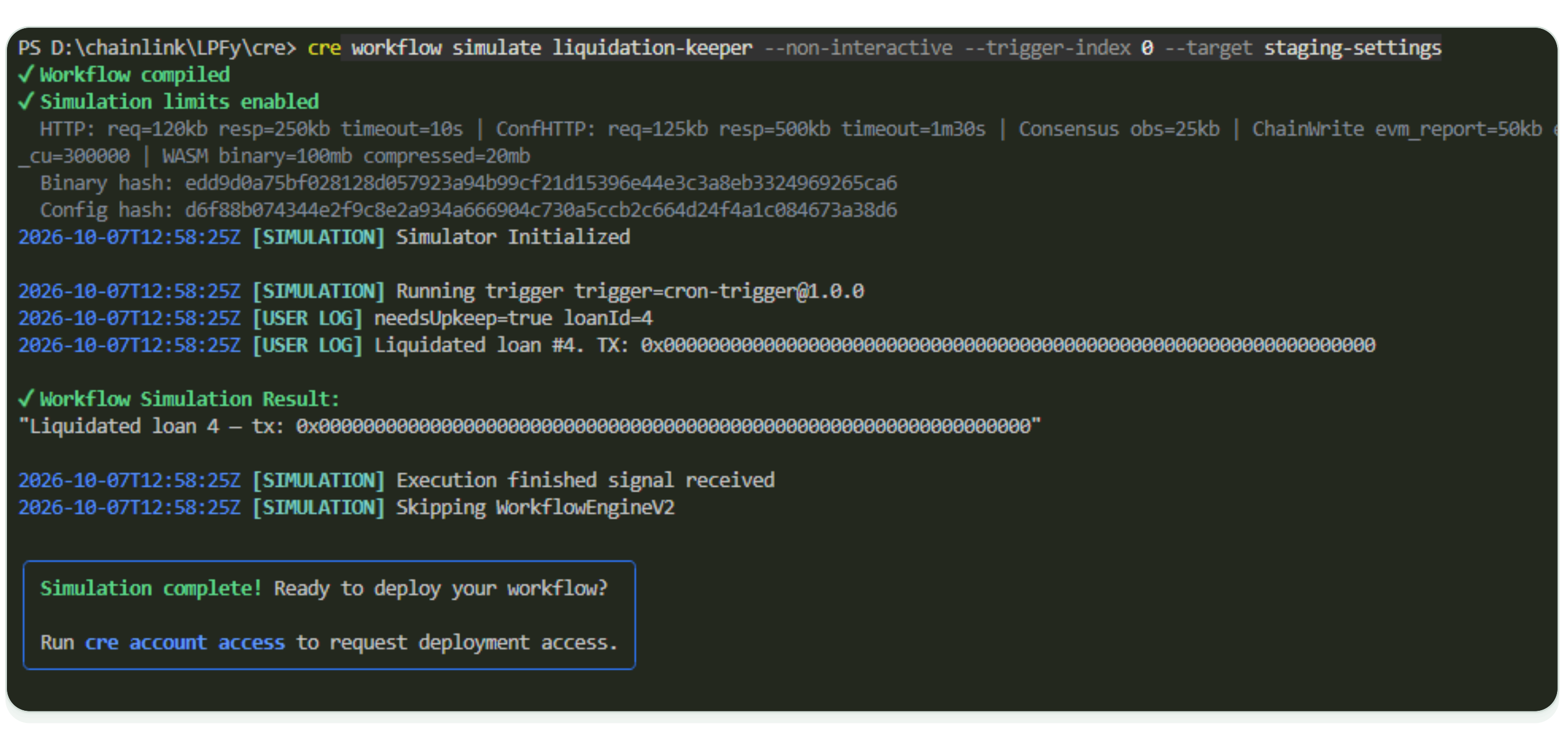
9. Hackathon demo script

1. **Setup loan:** Lender supplies Circle USDC → borrower posts Uniswap V3/V4 LP NFT → borrows near max LTV.
2. **Make liquidatable:** Admin lowers live `defaultLiquidationThresholdBps` (or wait for a price move) so `isLiquidatable(loanId)` is true.
3. **Dry-run CRE:**

```
cd cre
cre workflow simulate liquidation-keeper --non-interactive --trigger-index 0 --target staging-settings
```

Expect `needsUpkeep=true` and a simulated write (no chain spend).

Command output:



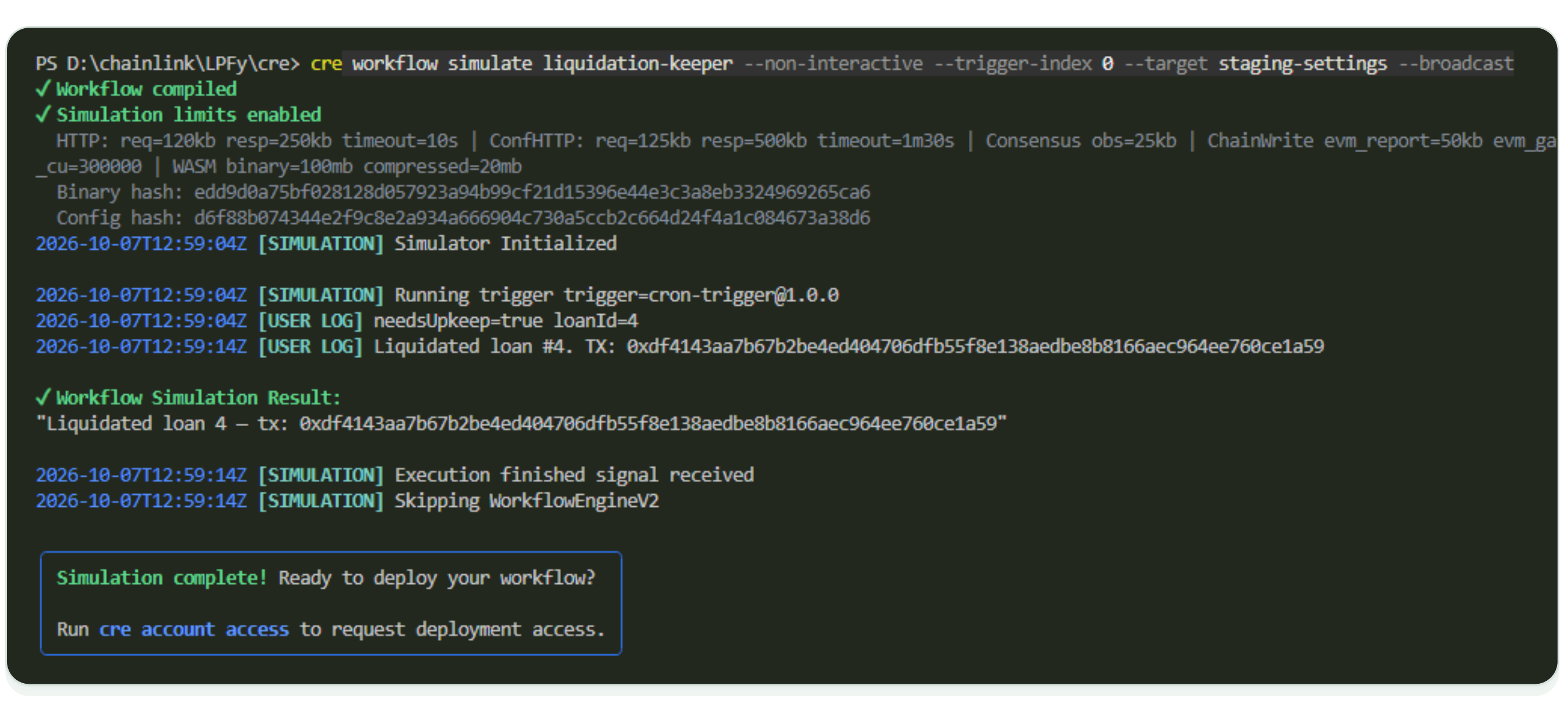
```
PS D:\chainlink\LPFy\cre> cre workflow simulate liquidation-keeper --non-interactive --trigger-index 0 --target staging-settings
✓ Workflow compiled
✓ Simulation limits enabled
  HTTP: req=120kb resp=250kb timeout=10s | ConfHTTP: req=125kb resp=500kb timeout=1m30s | Consensus obs=25kb | ChainWrite evm_report=50kb e
_cu=300000 | WASM binary=100mb compressed=20mb
  Binary hash: edd9d0a75bf028128d057923a94b99cf21d15396e44e3c3a8eb3324969265ca6
  Config hash: d6f88b074344e2f9c8e2a934a666904c730a5ccb2c664d24f4a1c084673a38d6
2026-10-07T12:58:25Z [SIMULATION] Simulator Initialized

2026-10-07T12:58:25Z [SIMULATION] Running trigger trigger=cron-trigger@1.0.0
2026-10-07T12:58:25Z [USER LOG] needsUpkeep=true loanId=4
2026-10-07T12:58:25Z [USER LOG] Liquidated loan #4. TX: 0x0000000000000000000000000000000000000000000000000000000000000000

✓ Workflow Simulation Result:
"Liquidated loan 4 — tx: 0x0000000000000000000000000000000000000000000000000000000000000000"

2026-10-07T12:58:25Z [SIMULATION] Execution finished signal received
2026-10-07T12:58:25Z [SIMULATION] Skipping WorkflowEngineV2

 Simulation complete! Ready to deploy your workflow?

 Run cre account access to request deployment access.
```

4. **Broadcast:**

```
cre workflow simulate liquidation-keeper --non-interactive --trigger-index 0 --target staging-settings --broadcast
```

Show explorer: Forwarder → `ReportProcessed(result=true)` → loan closed, empty NFT returned.

Command output:



```
PS D:\chainlink\LPFy\cre> cre workflow simulate liquidation-keeper --non-interactive --trigger-index 0 --target staging-settings --broadcast
✓ Workflow compiled
✓ Simulation limits enabled
  HTTP: req=120kb resp=250kb timeout=10s | ConfHTTP: req=125kb resp=500kb timeout=1m30s | Consensus obs=25kb | ChainWrite evm_report=50kb evm_ga
_cu=300000 | WASM binary=100mb compressed=20mb
  Binary hash: edd9d0a75bf028128d057923a94b99cf21d15396e44e3c3a8eb3324969265ca6
  Config hash: d6f88b074344e2f9c8e2a934a666904c730a5ccb2c664d24f4a1c084673a38d6
2026-10-07T12:59:04Z [SIMULATION] Simulator Initialized

2026-10-07T12:59:04Z [SIMULATION] Running trigger trigger=cron-trigger@1.0.0
2026-10-07T12:59:04Z [USER LOG] needsUpkeep=true loanId=4
2026-10-07T12:59:14Z [USER LOG] Liquidated loan #4. TX: 0xdf4143aa7b67b2be4ed404706dfb55f8e138aedbe8b8166aec964ee760ce1a59

✓ Workflow Simulation Result:
"Liquidated loan 4 — tx: 0xdf4143aa7b67b2be4ed404706dfb55f8e138aedbe8b8166aec964ee760ce1a59"

2026-10-07T12:59:14Z [SIMULATION] Execution finished signal received
2026-10-07T12:59:14Z [SIMULATION] Skipping WorkflowEngineV2

 Simulation complete! Ready to deploy your workflow?

 Run cre account access to request deployment access.
```

> Docs: Chainlink CRE · Data Feeds · Simulating workflows

10. Security model

CALLER	CAN LIQUIDATE?
Market owner (admin UI)	Yes
CreLiquidationReceiver (via CRE Forwarder)	Yes — authorized liquidator
Random EOA	No — `NotLiquidationAuthority`
Anyone calling `onReport` directly	No — must be Forwarder

Quick recap

- **Data Feeds** → fair LP collateral value → LTV & liquidation health.
- **CRE** → scheduled keeper → signed report → authorized on-chain liquidate.
- **Uniswap** → LP NFT collateral + unwind/swaps on liquidation.
- **PairVault** → recovered debt returns to ERC-4626 lenders; fee split optional.

Deployed contracts (Sepolia)

CRE keeper `config.staging.json` / `config.production.json` `contractAddress` must match `CreLiquidationReceiver` below. Source: `smart-contracts/deployments/sepolia-markets.json`.

CONTRACT	ADDRESS
`MarketLendingModule`	`0x618a17C71b1D3922172E1b77D6e3D17704DE9f41`
`CreLiquidationReceiver`	`0x32866b81408d0260259BC0109b9F0a300Cdca132`
`V3Adapter` / `V4Adapter`	`0x48ee590E073f5ae2FDFCEA465e6281115308dDc9` / `0x646DB0572422424947262fC3Bda2940f51504759`
PairVault USDC/WETH	`0xE47ABe4b88Fa2F47f4913710e8ecfBDE1205A5dc`
KeystoneForwarder (external)	`0x15fC6ae953E024d975e77382eEeC56A9101f9F88`
SwapRouter02 (external)	`0x3bFA4769FB09eefC5a80d6E87c3B9C650f7Ae48E`
